D213 Advanced Data Analytics

NLM3 Sentiment Analysis Using Neural Networks (Task 2)

Performance Assessment

Hillary Osei (Student ID #011039266)

Western Governors University, College of Information Technology

Program Mentor: Dan Estes

August 26, 2025

Table of Contents

Part I: Research Question

Section A1. Proposal of Question

The research question addressed is "Can reviews be automatically classified as positive or negative using a neural network trained on IMDB data?"

Section A2. Objectives or Goals

The objective of this analysis is to develop a sentiment classification model for IMDB reviews that distinguishes positive from negative opinions. The classification model will help studios and streaming platforms to monitor audience sentiment at scale, compare titles and campaigns over time, and identify emerging issues or strengths. Insights from the model will support data-driven decisions in marketing, content strategy, and product experience.

Section A3. Prescribed Network

Multiple neural network families can perform text classification, such as CNNs, RNNs, Transformers. For this analysis, recurrent neural networks will be used to classify IMDB reviews as positive or negative. RNN is built for ordered text sequences, reading each review left-to-right and right-to-left to capture context, word order, and local cues like negation ("not good") that changes sentiment (Wikipedia, n.d.). It also retains information over longer spans within a review, so phrases that modify earlier words are reflected in the final prediction. (Rahman, 2023)

RNN is commonly used for NLP tasks such as content moderation (Gongane et al., 2022) and sentiment analysis (Angelou, 2023). On IMDB data, RNN is well-suited to learn sentiment patterns (such as contrast words like "but"/"however") and to combine them with surrounding context, enabling useful text-classification predictions that generalize beyond exact word matches to the way words are used in sentences.

This project uses a recurrent neural network with a GRU layer to model ordered word sequences in IMDB reviews. GRUs maintain a hidden state across tokens, allowing the model to use context and word order to infer sentiment without needing a large compute. (Pandya, 2023)

Part II: Data Preparation

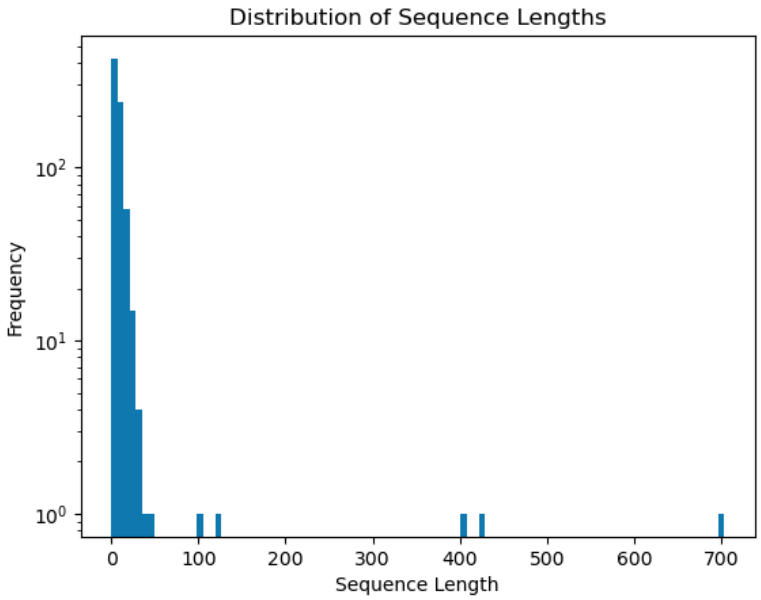
Section B1. Data Exploration

Across the 748 IMdB reviews (386 positive reviews, 362 negative reviews), there are 26 unique characters (a-z), 10 unique digits (0-9) and 21 unique special characters present. Frequent usual characters were punctuations such as en and em dashes, ellipsis (\x96, \x97, \x85) and accented letters (e.g., é, å). There were no emojis found in the dataset.

After removing unusual characters and stopwords, converting reviews to all lowercase, tokenizing, and lemmatizing reviews, a Keras Tokenizer is fit on the cleaned reviews, resulting in a vocabulary size of 2,179 unique words. Vocabulary size is the total number of unique words present in a dataset.

A word embedding length of 7.0 is proposed, taken from the median review length of 7 tokens. This is ideal because of the short-length of most of the reviews in the dataset.

Visualizing the sequence lengths shows a strong right skew. The median is at 7 tokens, with few very long reviews. To avoid truncating any text in the reviews, the maximum sequence length is set to the longest review observed, at 703 tokens. Shorter reviews are later padded to this length. This choice is supported by the observed distribution and also preserved all the words in the dataset at the cost of doing extra padding for shorter-length reviews.



Section B2. Tokenization

Tokenization is a preprocessing step that converts text strings into smaller units, known as tokens, so language can be processed by a model. The goal of tokenization is to represent text in a way that can be meaningfully understood by a machine while still retaining context. (Awan, 2024) Another goal is to make text easier to analyze. When data is broken down into smaller units, it becomes more manageable. In addition, tokenization also helps with identifying and removing stopwords and unusual/unnecessary characters.

Below is code showing how tokenization was used in the analysis (removing unusual characters, find number of unique words in original dataframe vs new dataframe for comparison).

```
# Removes unusual characters + converts reviews to all lowercase, tokenizes & lemmatizes reviews, & removes stopwords

# Create an empty list to store cleaned reviews
description_list = []

# Define the list of stopwords, set language to English
stop_words = stopwords.words('english')

# Remove special characters
for description in imdb['review']:
    description = re.sub("[^a-zA-Z]", " ", description)
# Convert to lower case
    description = description.lower()
# Perform tokenization
    description = nltk.word_tokenize(description)
# Perform lemmatization
    lemma = nltk.WordNetLemmatizer()
    description = [lemma.lemmatize(word) for word in description]
# Remove stopwords
    description = [word for word in description if not word in stop_words]
    description = " ".join(description)
    description_list.append(description)
```

```
[12]: tokenizer = Tokenizer()
      tokenizer.fit_on_texts(description_df['clean_reviews'])
      print("Number of unique words:", len(tokenizer.word_index) + 1)

      Number of unique words: 2719
```

```
[16]: # Tokenization of original dataframe for comparison
      tokenizer.fit_on_texts(imdb['review'])
      word_count = len(tokenizer.word_index) + 1
      print("Number of unique words:", word_count)

      # Print imdb tokens
      imdb['review'].apply(word_tokenize)

      Number of unique words: 3342

[16]: 0      [A, very, ,, very, ,, very, slow-moving, ,, ai...
      1      [Not, sure, who, was, more, lost, -, the, flat...
      2      [Attempting, artiness, with, black, &, white, ...
      3      [Very, little, music, or, anything, to, speak,...
      4      [The, best, scene, in, the, movie, was, when, ...
                                 ...
      743    [I, just, got, bored, watching, Jessice, Lange...
      744    [Unfortunately, ,, any, virtue, in, this, film...
      745            [In, a, word, ,, it, is, embarrassing, .]
      746                              [Exceptionally, bad, !]
      747    [All, in, all, its, an, insult, to, one, 's, i...
      Name: review, Length: 748, dtype: object
```

```
[17]: # Tokenization of description_df
tokenizer = Tokenizer()
tokenizer.fit_on_texts(description_df['clean_reviews'])
print("Number of unique words:", len(tokenizer.word_index) + 1)

# Print tokens
description_df['clean_reviews'].apply(word_tokenize)
```

```
Number of unique words: 2719
```

```
[17]: 0      [slow, moving, aimless, movie, distressed, dri...
1      [sure, wa, lost, flat, character, audience, ne...
2      [attempting, artiness, black, white, clever, c...
3                        [little, music, anything, speak]
4      [best, scene, movie, wa, gerardo, trying, find...
                             ...
743    [got, bored, watching, jessice, lange, take, c...
744    [unfortunately, virtue, film, production, work...
745                                [word, embarrassing]
746                                [exceptionally, bad]
747      [insult, one, intelligence, huge, waste, money]
Name: clean_reviews, Length: 748, dtype: object
```

Section B3. Padding Process

After tokenization, the IMdB reviews vary in length so sequences were standardized before modeling using Tensorflow + Keras pad_sequences. Padding was applied after each sequence (post-padding) , meaning zeros were added to the end of the token list until a fixed target length was reached. The target length was set to 703, the longest review observed in the dataset, therefore no truncation was needed. Shorter reviews were padded while the longest reviews remained the same.

```
[18]: from tensorflow.keras.preprocessing.sequence import pad_sequences

# (DataCamp, Machine Translation with Keras, n.d.)

# Pad sequences
sequences = tokenizer.texts_to_sequences(description_list)
padded_sequences = pad_sequences(sequences, maxlen=review_max, padding='post')

print(padded_sequences)
```

```
[[ 197  304 1040 ...    0    0    0]
 [ 417    3  306 ...    0    0    0]
 [1044 1045  102 ...    0    0    0]
 ...
 [ 143  468    0 ...    0    0    0]
 [2718    5    0 ...    0    0    0]
 [ 398    4  380 ...    0    0    0]]
```

Section B4. Categories of Sentiment

The dataset's sentiments are 0 (representing negative sentiment) and 1 (representing positive sentiment). Therefore, two categories of sentiment are used in the analysis. Sigmoid was selected for the activation function for the final dense layer of the network. It maps input values any value between 0 and 1 (Vishwakarma, 2024) and makes it suitable for binary classification.

Section B5. Steps to Prepare the Data

1. Imported necessary libraries and packages for analysis (pandas/numpy for data handling, matplotlib for plots, TensorFlow/Keras for tokenization, padding, and modeling, etc)
2. Load IMdB dataset and set labels (review, rating)
3. Do exploratory data analysis: Inspect value counts of positive and negative reviews in the dataset, find unusual characters, determine word embedding length, visualize sequence length
4. Clean data: lowercased text, removed punctuation and special symbols, removed whitespaces, lemmatize and use stop-word filtering to standardize wording of reviews; create new dataframe (description_df) to hold clean data
5. Calculate total vocabulary size (and compare with original dataframe vs cleaned dataframe)
6. Determine proposed word embedding length, proposing a number of 7
7. Calculate the maximum sequence length by getting the maximum length of reviews and chose 703 (the longest review observed)
8. Tokenize data by fitting Keras Tokenizer on cleaned review
9. Pad data to ensure sequences are the same length, applying post-padding with the value of 0 up to the 703 (maximum sequence length)
10. Define sentiment categories, modeling sentiment as two classes (negative = 0, positive = 1), using sigmoid as activation function
11. Instantiate model with embedding layer and dense layers
12. Set x and y to prepare for splitting data. X is padded, Y is the values from description_df's values
13. Split data into 80/20 train/test sets to evaluate performance. The split was set up in this ratio as it is the standard when building a model. There is a 30% validation split applied to the training fold during fitting (419 train, 179 validation, 150 testing). 30% was chosen since it is common practice for smaller datasets.

The size and shape of the sets are:
X_train shape: (598, 703) and size: 420394
X_test shape: (150, 703) and size: 105450
y_train shape: (598,) and size: 598
y_test shape: (150,) and size: 150

Section B6. Prepared Data Set

Prepared dataset is attached "d213-task2.csv," as well as the training and testing sets "X_train.csv," "X_test.csv," "Y_train.csv," "Y_test.csv."

Part III: Network Architecture

Section C1. Model Summary

Below is the output of the model summary from TensorFlow:

```
# Create model
from keras.models import Sequential
from keras.layers import Embedding, Dense, GRU
from keras import backend
from tensorflow.keras.callbacks import EarlyStopping

activation = 'sigmoid'
loss = 'binary_crossentropy'
optimizer = 'adam'
num_epochs = 20
embedding_dim = 7

# Define early stopping monitor
early_stopping_monitor = EarlyStopping(patience=2)

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=word_count+1,
                              output_dim=embedding_dim,
                              input_length=review_max),
    tf.keras.layers.GRU(128),
    tf.keras.layers.Dense(100, activation='relu'),
    tf.keras.layers.Dense(50, activation='relu'),
    tf.keras.layers.Dense(1, activation=activation)
])

model.compile(loss=loss,optimizer=optimizer,metrics=['accuracy'])
_ = model(tf.zeros((1, review_max), dtype=tf.int32))

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(1, 703, 7)	23,401
gru_1 (GRU)	(1, 128)	52,608
dense_4 (Dense)	(1, 100)	12,900
dense_5 (Dense)	(1, 50)	5,050
dense_6 (Dense)	(1, 1)	51

Total params: 94,010 (367.23 KB)
Trainable params: 94,010 (367.23 KB)
Non-trainable params: 0 (0.00 B)

Section C2. Network Architecture

The network contains five layers. The first is the Embedding layer that stores word vectors and accounts for most of the parameters with 23,401 trainable parameters (GeeksforGeeks, 2025). The second layer is GRU (the recurrent layer), which has 52,608. This is followed by three dense layers. The first one is a 100-unit layer with 12,900 parameters, the second one is a 50-unit layer with 5,050 parameters, and the third one has 1 unit and 51 parameters. In total, the model has 94,010 parameters, all of which are trainable.

Section C3. Hyperparameters

The network uses several hyperparameters for binary sentiment classification. The embedding dimension is fixed to 7 to match the proposed word embedding length, to keep the model small and reduce overfitting on short reviews. The GRU layer uses 128 units, which gives enough capacity to capture review context and word order while also being quick to train. The first two dense layers use ReLU, which trains quickly (Liu, 2017) and helps models learn non-linear patterns (Lopin, 2019). The final dense layer uses sigmoid activation so that the output is a probability between 0 and 1 for the sentiment classes. The first dense layer contains 100 nodes, the second layer 50 nodes, and the third layer has 1 node. Binary crossentropy is chosen as the loss function due to the fact that the review values are binary (0 for negative and 1 for positive sentiment). Adam is the optimizer used for the model. It is commonly used because it is easy to implement and is efficient (Agarwal, 2023) Stopping criteria is the condition in which the optimization of a model can terminate. (ScienceDirect, n.d.) EarlyStopping is used for stopping criteria with patience being set to 2. This means that during training, the model won't run more than 2 epochs without an improvement. The quality of the model is evaluated by measuring the accuracy on both the training and test sets. The training accuracy is ~52.1% and the testing accuracy is ~49.3%.

```
[30]: # Get model accuracy scores
testing_score = model.evaluate(X_test, y_test, verbose=0)
print('Test accuracy:', {testing_score[1]})

training_score = model.evaluate(X_train, y_train, verbose=0)
print('Train accuracy:', {training_score[1]})

Test accuracy: {0.4933333396911621}
Train accuracy: {0.52173912525177}
```

Part IV: Model Evaluation

Section D1. Stopping Criteria

Early Stopping is used as the stopping criteria to prevent overfitting. The patience is set to 2, meaning the model won't run more than 2 consecutive times if there is no improvement. The number of epochs is set to 20, meaning the model will run a maximum of 20 iterations before it stops. If the stopping criteria is met before 20 epochs are run, the model will stop. Below, it shows that the model was able to stop before 20 epochs, terminating at epoch 8. Validation accuracy plateaued around 0.5444 and validation loss stayed around 0.69.

```
[29]: # Train model
result = model.fit(X_train, y_train, epochs=20, batch_size=50, validation_split=0.3,
                   callbacks=[early_stopping_monitor], verbose=True)
```

```
Epoch 1/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 29s 2s/step - accuracy: 0.4545 - loss: 0.6934 - val_accuracy: 0.5444 - val_loss: 0.6928
Epoch 2/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 15s 2s/step - accuracy: 0.5120 - loss: 0.6933 - val_accuracy: 0.5444 - val_loss: 0.6922
Epoch 3/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 19s 2s/step - accuracy: 0.5120 - loss: 0.6930 - val_accuracy: 0.5444 - val_loss: 0.6924
Epoch 4/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 16s 2s/step - accuracy: 0.5120 - loss: 0.6930 - val_accuracy: 0.5444 - val_loss: 0.6919
Epoch 5/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 12s 1s/step - accuracy: 0.5120 - loss: 0.6930 - val_accuracy: 0.5444 - val_loss: 0.6919
Epoch 6/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 15s 2s/step - accuracy: 0.5120 - loss: 0.6929 - val_accuracy: 0.5444 - val_loss: 0.6917
Epoch 7/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 15s 2s/step - accuracy: 0.5120 - loss: 0.6929 - val_accuracy: 0.5444 - val_loss: 0.6918
Epoch 8/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 15s 2s/step - accuracy: 0.5120 - loss: 0.6930 - val_accuracy: 0.5444 - val_loss: 0.6920
```

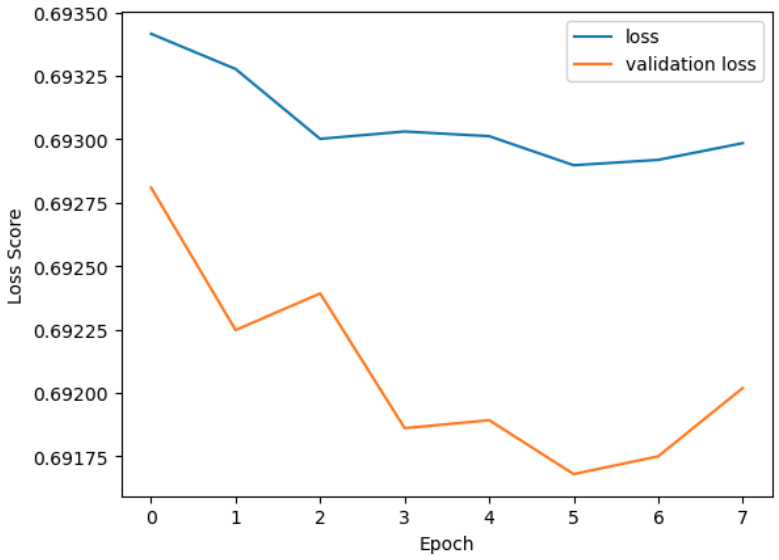
Section D2. Fitness

The fitness of the model is determined using accuracy scores. The testing accuracy score is at 49.3% and the training accuracy score is at 52.1%. Overfitting is addressed by using a stopping criteria, with patience being set to 2.

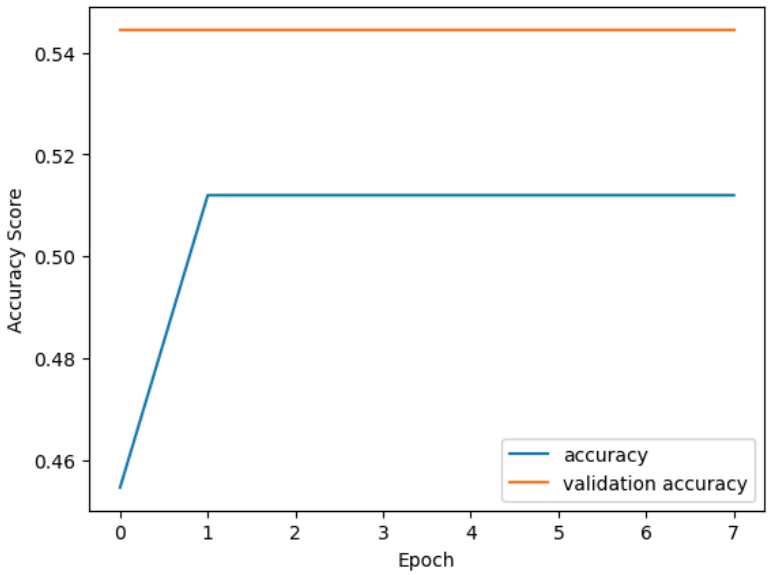
Section D3. Training Process

Below are visualizations of the loss and accuracy line graphs to show model training process:

Loss:



Accuracy:



Section D4. Predictive Accuracy

The predictive accuracy of the model is the test accuracy score, calculated to be at 0.493 or 49.3%. This means that the model correctly predicts a viewer's sentiment of 49.3% of the text inputs given.

Part V: Summary and Recommendations

Section E. Code

The trained network within the neural network was saved with the code below:

```
# Save trained network
model.save('d213-model.h5')
```

Section F. Functionality

The IMDB review dataset used in the analysis consists of 748 rows,or 748 film reviews. The model is trained on 598 reviews (which is 80% of the dataset) and tested on 150 reviews (which is 20% of the dataset). The model accurately predicts the sentiment for 52% of the reviews in the training dataset and the model accurately predicts sentiment for 49% of the review in the testing dataset. The architecture of the neural network model, its settings, embedding size, etc likely have an influence on the accuracy scores.

Section G. Recommendations

Given the results, 52% accuracy on the training data and 49% on the testing dataset, the current model does not reliably classify IMDB reviews as positive or negative. Performance of the model is close to chance, so should not be used for decision-making. The film/movie industry can use these findings as a baseline. To better improve the model, more labeled reviews across genres can be collected and a clear accuracy target. In the meantime, the current model can be used for tracking trends in sentiment over time.

Part VI: Reporting

Section H. Reporting

Jupyter notebook (in html format) is attached.

Section I. Sources for Third-Party Code

Machine Translation with Keras | Part 2: Preprocessing the text [Slides].

https://campus.datacamp.com/pdf/web/viewer.html?file=https://projector-video-pdf-converter.datacamp.com/17605/chapter3.pdf#page=12

Section J. Sources

Agarwal, R. (2023, September 13). *Complete Guide to the Adam Optimization Algorithm*. Built In. Retrieved August 24, 2025, from https://builtin.com/machine-learning/adam-optimization

Angelou, R. (2023, August 27). *Recurrent Neural Network: Working, Applications, Challenges | by Ambika | **AI monks**.**io***. Medium. Retrieved August 23, 2025, from https://medium.com/aimonks/recurrent-neural-network-working-applications-challenges-f445f5f297c9

Awan, A. A. (2024, November 22). *What is Tokenization? Types, Use Cases, Implementation*. DataCamp. Retrieved August 24, 2025, from https://www.datacamp.com/blog/what-is-tokenization

GeeksforGeeks. (2025, July 23). *What is Embedding Layer ?* GeeksforGeeks. Retrieved August 24, 2025, from https://www.geeksforgeeks.org/deep-learning/what-is-embedding-layer/

Gongane, V. U., Munot, M. V., & Anuse, A. D. (2022, September 5). *Detection and moderation of detrimental content on social media platforms: current status and future directions*. NIH NLM. pmc.ncbi.nlm.nih.gov/articles/PMC9444091/

Liu, D. (2017, November 29). *A Practical Guide to ReLU. Start using and understanding ReLU… | by Danqing Liu*. Medium. Retrieved August 24, 2025, from https://medium.com/@danqing/a-practical-guide-to-relu-b83ca804f1f7

Lopin, M. (2019, October 22). *Why is ReLU non-linear?. ReLU doesn't look very non-linear, but… | by Maxim Lopin*. Medium. Retrieved August 24, 2025, from https://medium.com/@maximlopin/why-is-relu-non-linear-aa46d2bad518

Pandya, K. (2023, May 4). *Understanding Gated Recurrent Unit (GRU) in Deep Learning*. Medium. Retrieved August 26, 2025, from https://medium.com/@anishnama20/understanding-gated-recurrent-unit-gru-in-deep-learning-2e54923f3e2

Rahman, M. A. (2023, March 5). *The Unseen Depths of RNN & LSTM: The Power of Deep Learning in NLP*. Medium. https://medium.com/@asifurrahmanaust/the-unseen-depths-of-rnn-lstm-the-power-of-deep-learning-in-nlp-5f29e04cd8ae

ScienceDirect. (n.d.). https://www.sciencedirect.com/topics/engineering/stopping-criterion

Vishwakarma, S. (2024, December 9). *Understand Sigmoid Function in Artificial Neural Networks*. Analytics Vidhya. Retrieved August 24, 2025, from https://www.analyticsvidhya.com/blog/2023/01/why-is-sigmoid-function-important-in-artificial-neural-networks/

Wikipedia. (n.d.). *Recurrent neural network - Wikipedia*. Wikipedia, the free encyclopedia. Retrieved August 23, 2025, from https://en.wikipedia.org/wiki/Recurrent_neural_network